

Flask AI Chat Application

A full-stack AI chat application built with Flask, featuring dual-AI failover (OpenAI + Google Gemini), encrypted message storage, real user authentication, a ChatGPT-style interface, structured error logging, an automated CI/CD pipeline, a scheduled ETL job, and a live Power BI analytics dashboard.

Built as a portfolio project to demonstrate practical skills across backend development, data engineering, security practices, and business intelligence.

🔗 **Live app:** flask-chatgpt-app.onrender.com

Hosted on a free tier — if it's been idle a while, the first load can take up to a minute while it wakes up.

Documentation

This repo includes three guides, each written for a different purpose:

Guide	Use this if you want to...
<u>DOCUMENTATION.html</u>	Understand how the whole project was built, step by step, from an empty folder to a live app — architecture, security, deployment, CI/CD, ETL, and analytics all explained in plain language
<u>SETUP_GUIDE.html</u>	Get your own copy of this project running on your own computer, with your own database and API keys
<u>TECHNICAL_DOC.md</u>	A concise technical reference — architecture diagram, database schema, tech stack, and design decisions

Features

- **AI Chat** with automatic failover — if OpenAI is unavailable (rate limits, no credits, downtime), the app silently falls back to Google Gemini
- **Real user accounts** — email/password signup and Google OAuth sign-in
- **ChatGPT-style interface** — collapsible sidebar, persistent conversation history per user, mobile-responsive layout
- **Encrypted message storage** — all chat content is encrypted at rest (Fernet/AES)
- **Live analytics dashboard** — a multi-page Power BI report connected via DirectQuery, auto-refreshing every 10 seconds, covering usage KPIs, AI model performance, question categorization, and system health
- **Structured error logging** — a global error handler captures unhandled exceptions app-wide, feeding directly into the dashboard

- **CI/CD pipeline** — GitHub Actions automatically runs tests on every push, before Render deploys the change live
- **ETL pipeline** — a scheduled daily job aggregates raw usage data into a summary table for faster analysis
- **Deployed live** — running on Render, backed by a Neon (serverless PostgreSQL) production database, with an independent local database used for development

Tech Stack

Layer	Technology
Backend	Python, Flask, Gunicorn
Database	PostgreSQL (local for dev, Neon for production), SQLAlchemy
AI Providers	OpenAI API, Google Gemini API
Auth	Flask-Login, Werkzeug (password hashing), Authlib (Google OAuth)
Encryption	<code>cryptography</code> (Fernet)
Frontend	HTML, CSS, vanilla JavaScript
Analytics	Power BI Desktop (DirectQuery, dual local/production connections)
CI/CD	GitHub Actions
Testing	pytest
Hosting	Render
Version control	Git, GitHub (private repo)

Project Structure

```
flask-chatgpt-app/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml          # Runs tests on every push
│   │   └── etl.yml         # Scheduled daily ETL job
│   └── tests/
│       └── test_app.py     # Automated smoke tests
├── app.py                  # Main Flask application
├── etl_daily_stats.py      # Standalone ETL script
├── templates/
│   ├── index.html         # Chat interface
│   └── login.html         # Sign in / sign up page
├── static/
│   ├── style.css
│   └── web/                # Icon assets
├── power-bi/
│   ├── flaskchatapp_analytics_dashboard.pbix # Local dev version
│   └── flaskchatapp_dashboard_LIVE_neon.pbix  # Live production version
├── requirements.txt
├── .env.example            # Template for required environment
                             variables
├── .gitignore
├── DOCUMENTATION.html
├── SETUP_GUIDE.html
└── TECHNICAL_DOC.md
```

Quick Start (for development)

```
bash

python -m venv venv
source venv/Scripts/activate # Windows Git Bash
pip install -r requirements.txt
python app.py
```

Requires a local PostgreSQL database and a `.env` file — see [SETUP_GUIDE.html](#) for the full walkthrough, including database setup and getting your own API keys.

Status

Complete and live. Core chat, dual-AI fallback, authentication, encryption, database logging, error monitoring, CI/CD, ETL, and the Power BI analytics dashboard are all built, deployed, and working end to end.

